

Name : Bisma Fatima

Class : CS- A 24

Roll No : F24605036

Course : Data Structure

Code : CS-210

Lecturer : Sir Mehran Yousaf

Assignment 03

Name: Bisma fatima

Question No 01:

Implement BST with all operations.

Program

```
#include <iostream>
```

```
using namespace std;
```

```
struct Node {  
    int data;  
    Node *left;  
    Node *right;  
};
```

```
class BST {
```

```
private:
```

```
    Node *root;
```

```
public:
```

```
    BST()
```

```
    { root = nullptr;
```

```
    }
```

```
    void insert(int val)
```

```
    {
```

```
        Node *Newnode = new Node();
```

```
        Newnode → data = val;
```

```
        Newnode → left = nullptr;
```

```
        Newnode → right = nullptr;
```

```
        if (root == nullptr)
```

```
        { root = Newnode; return; }
```

```
Node *temp = root;
```

```
while (true)
```

```
{ if (val < temp->data)
```

```
{ if (temp->left == NULLPtr)
```

```
{ temp->left = Newnode;
```

```
break;
```

```
temp = temp->left;
```

```
}
```

```
else
```

```
{ if (temp->right == NULLPtr)
```

```
{ temp->right = Newnode;
```

```
break;
```

```
temp = temp->right;
```

```
}
```

```
}
```

```
}
```

```
bool search(int val)
```

```
{
```

```
Node *temp = root;
```

```
while (temp != NULLPtr)
```

```
{
```

```
if (temp->data == val)
```

```
return true;
```

```
else if (temp->data > val)
```



```

        temp = temp → left;
    else
        temp = temp → right;
    }
    return false;
}

```

```

void inorder (Node * node)
{
    if (node != nullptr)
    {
        inorder (node → left);
        cout << node → data;
        inorder (node → right);
    }
}

```

```

void printInorder()
{
    inorder (root);
    cout << endl;
}

```

```

void preorder (Node * node)
{
    if (node != nullptr)
    {
        cout << node → data;
        preorder (node → left);
        preorder (node → right);
    }
}

void printPreorder()
{
    preorder (root); cout << endl; }

```



```

void postorder(Node *node)
{
    if (node != nullptr)
    {
        postorder(node → left);
        postorder(node → right);
        cout << node → data;
    }
}

```

```

void printPostorder()
{
    postorder(root);    return
    cout << endl;
}

```

```

void delete(int val)
{
    root = deleteNode(root, val);
}

```

```

Node * deleteNode(Node * node, int val)
{
    if (node == nullptr)
        return Node;

    if (val < node → data)
    {
        node → left = deleteNode(node → left, val);
    }
    else if (val > node → data)
    {
        node → right = deleteNode(node → right, val);
    }
    else {
        if (node → left == nullptr)
        {
            Node *temp = node → right;

```



```
delete node;
```

```
    return temp;
}
```

```
else if ( node → right == nullPtr)
```

```
{
```

```
    Node *temp = node → left;
```

```
    delete the node;
```

```
    return temp;
}
```

```
Node *temp = minvalue(node → right);
```

```
node → data = temp → data;
```

```
node → right = deleteNode(node → right, temp  
                        → data);
```

```
}
```

```
return node;
```

```
}
```

```
Node * minvalue ( Node * node)
```

```
{ while ( node → left != nullPtr)
```

```
    node = node → left;
```

```
return node;
```

```
}
```

```
};
```

```
int main()
```

```
{
```

```
    Bst tree ;
```

```
    tree.insert(50);
```

```
    tree.insert(30);
```

```
    tree.insert(70);
```


insert .

```
tree.insert(20);
```

```
tree.insert(40);
```

```
tree.insert(60);
```

```
tree.insert(80);
```

```
cout<< "Inorder: " ;
```

```
tree.printInorder();
```

```
cout<< "Preorder: "; tree.printPreorder();
```

```
cout<< "Postorder: "; tree.printPostorder();
```

```
cout<< "Search 40: " << (tree.search(40) ? "found":
```

```
" Not found");
```

```
tree.delete(20);
```

```
cout<< "Inorder after deleting 20: ";
```

```
tree.printInorder();
```

```
return 0;
```

```
}
```

Output

Inorder: 20 30 40 50 60 70 80

Preorder: 50 30 20 40 70 60 80

Postorder: 20 40 30 60 80 70 50

Search 40: Found

Inorder after deleting 20: 30 40 50 60 70 80.